

FastAPI & Backend Development

Interview Revision Handout — Section 2: Setup, Routing & Documentation

PART 1 — ENVIRONMENT SETUP

1. Python Installation

FastAPI needs **Python 3.8+** (3.10+ recommended for best type-hint features like `int | None`). Check version and install via python.org or a version manager (pyenv).

```
python --version
python3 --version
```

2. Virtual Environment

A **virtual environment** is an isolated Python environment with its own installed packages, separate from the system-wide Python. It prevents dependency/version conflicts between projects.

```
python -m venv venv          # create
source venv/bin/activate     # activate (Mac/Linux)
venv\Scripts\activate       # activate (Windows)
deactivate                  # exit env
```

■ **Interview Q:** Why use a venv? — Different projects may need different package versions (e.g. FastAPI 0.95 vs 0.110); a venv isolates each project's dependencies so they never clash.

3. pip install

pip is Python's package manager. It installs packages from PyPI into the active environment.

```
pip install fastapi uvicorn[standard]
pip install -r requirements.txt      # install from a list
pip freeze > requirements.txt       # export installed packages
```

4. Editor / IDE Setup

- **VS Code** — lightweight, fast, huge extension ecosystem; most commonly used for FastAPI projects.
- **PyCharm** — full-featured Python IDE with deeper built-in refactoring, debugging, and Django/DB tooling — heavier but very powerful.

Useful VS Code Extensions

Extension	Purpose
Python (Microsoft)	Core language support — IntelliSense, debugging, environments
Ruff	Extremely fast linter/formatter (replaces flake8 + black + isort in one tool)
Error Lens	Shows errors/warnings inline next to the code, instead of only in the Problems tab
Pylance	Fast type checking & rich autocompletion (bundled with Python extension)

■ **Interview Q:** VS Code vs PyCharm — VS Code is preferred for speed and flexibility across stacks; PyCharm is preferred for large, complex Python-only projects needing advanced refactoring/debugging out of the box.

PART 2 — ENDPOINTS & ROUTING

1. API Endpoint (recap)

An **endpoint** is a unique combination of a **URL path** and an **HTTP method** that triggers specific server logic. Example: `GET /products/{id}` and `DELETE /products/{id}` are two different endpoints on the same path.

2. Decorator Concept (recap)

A **decorator** (`@app.get(...)`, `@app.post(...)`, etc.) wraps a plain Python function and registers it as the handler for a route, without modifying the function body itself. It's Python's clean way to attach metadata/behaviour to a function.

```
@app.get("/items/{item_id}")
def read_item(item_id: int):
    return {"item_id": item_id}
```

3. Relative vs Absolute vs Nested Paths

Type	Example	Meaning
Absolute path	https://api.site.com/users/5	Full URL including scheme + domain — used outside the app (in browsers, cli)
Relative path	/users/5	Path only, relative to the current host — this is what you define in @app.get()
Nested path	/users/5/orders/12	Path expressing a parent-child resource relationship (order 12 belongs to user 5)

```
@app.get("/users/{user_id}/orders/{order_id}")
def get_order(user_id: int, order_id: int):
    return {"user_id": user_id, "order_id": order_id}
```

■ **Interview Q:** Why use nested paths? — They clearly model resource hierarchy/ownership (an order always belongs to a user), improving API readability and REST design.

4. Starting the Server

FastAPI apps run via an **ASGI server**, typically **Uvicorn**.

```
uvicorn main:app
# main = filename (main.py)
# app = FastAPI() instance name inside that file
```

5. Development Mode (auto-reload)

Running with `--reload` watches your files and automatically restarts the server on code changes — used only in development, never in production (it adds overhead and file-watching isn't safe for prod).

```
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

■ **Interview Q:** Why avoid `--reload` in production? — It uses a file watcher and single-process reloader meant for fast local iteration, not stability/performance; production uses multiple workers (e.g. via Gunicorn+Uvicorn workers) without `--reload`.

PART 3 — API DOCUMENTATION & TESTING TOOLS

1. Automatic API Documentation

FastAPI auto-generates interactive documentation from your route definitions, type hints, and Pydantic models — based on the **OpenAPI specification** (formerly Swagger spec). No manual doc writing needed.

Route	Tool	Description
/docs	Swagger UI	Interactive docs — you can expand each endpoint and "Try it out" (send real requests) directly in the browser
/redoc	ReDoc	Clean, read-only, three-panel documentation — better for sharing/reading, not for testing
/openapi.json	OpenAPI schema	Raw JSON schema describing the whole API — used by tools like Postman to auto-import endpoints

■ **Interview Q:** What is Swagger? — Swagger is the original tool/spec that evolved into the OpenAPI Specification; "Swagger UI" is the interactive doc renderer FastAPI serves at /docs.

2. Scaling FastAPI

Because FastAPI apps are ASGI apps, they scale by running **multiple worker processes** (each with its own event loop) behind a process manager/load balancer.

- **Multiple Uvicorn workers** — e.g. `gunicorn -k uvicorn.workers.UvicornWorker -w 4 main:app`
- **Horizontal scaling** — run multiple containers/instances behind a load balancer (common in Docker/Kubernetes setups)
- **Async I/O** — using async DB drivers/HTTP clients so a single worker can juggle many concurrent requests without blocking
- **Caching & connection pooling** — reduce repeated DB/network load at scale

3. Postman

Postman is a GUI tool for manually sending HTTP requests and inspecting responses — used to test API endpoints during development, independent of a frontend. It supports collections (saved groups of requests), environments (variables like base URL/token), and can even import an API directly from its OpenAPI/Swagger schema.

- Test endpoints with custom headers, body, and auth without writing frontend code.
- Save request collections to share with a team or re-run for regression testing.
- Import /openapi.json directly to auto-generate a full request collection.

■ **Interview Q:** *Postman vs /docs (Swagger UI) — Both let you send test requests, but Postman offers scripting, environments, collections, and automated test suites; Swagger UI is quicker for a one-off manual check straight from the code.*